

Skill 2

Process Abstraction, `fork()`, `exec()` and Process Tree

Aim

To understand process abstraction in Linux, create processes using `fork()`, study the `exec()` family of system calls, analyze parent-child relationships, inspect the process tree and practice system call tracing.

Requirements

- Ubuntu Linux
- GCC
- Terminal
- C compiler

Step-by-Step Implementation

Step 1: Create the C Program

Create a file:

```
nano src/process_demo.c
```

Step 2: Include Required Header Files

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
```

Step 3: Create a Child Process

The `fork()` system call is used to create a new child process.

```
pid_t pid = fork();
```

Step 4: Check the Process

```
if (pid < 0) {
    perror("fork failed");
}
else if (pid == 0) {
    printf("Child Process\n");
    printf("Child PID: %d\n", getpid());
}
else {
    printf("Parent Process\n");
    printf("Parent PID: %d\n", getpid());
    wait(NULL);
}
```

Step 5: Compile the Program

```
gcc src/process_demo.c -o process_demo
```

Step 6: Execute

```
./process_demo
```

Step 7: Understand **exec()**

The **exec()** family replaces the currently running process with another program.

Example:

```
execl("/bin/ls", "ls", NULL);
```

Unlike **fork()**, **exec()** does not create another process. It replaces the existing process image.

Step 8: Inspect Processes

```
ps
```

For a detailed process tree:

`pstree`

Step 9: Trace System Calls

Install `strace`:

```
sudo apt install strace
```

Run:

```
strace ./process_demo
```

Observation

The `fork ( )` system call created a child process from the parent process. The parent and child processes had different process IDs. The `exec ( )` family was observed to replace a process with another executable program. The process hierarchy was inspected using `pstree`, and system calls were observed using `strace`.

End Result

The concepts of process abstraction, process creation, process replacement and parent-child relationships were successfully demonstrated and analyzed.